

CSE2008 – Operating Systems

Dr. Venkata Rami Reddy Ch
Sr.Assistant Professor
School of Computer Science & Engineering
VIT-AP University

- What operating systems do,
- Computer System Organization,
- Operating system operations – Process Management, Memory Management, Storage Management, I/O Systems, Protection & Security,
- Types of Operating Systems – Batch O.S., Multi programmed O.S., Time Sharing O.S., Distributed O.S., Virtualization, Real Time Embedded Systems, Open source O.S.,
- System calls & Overview on Types of System calls.

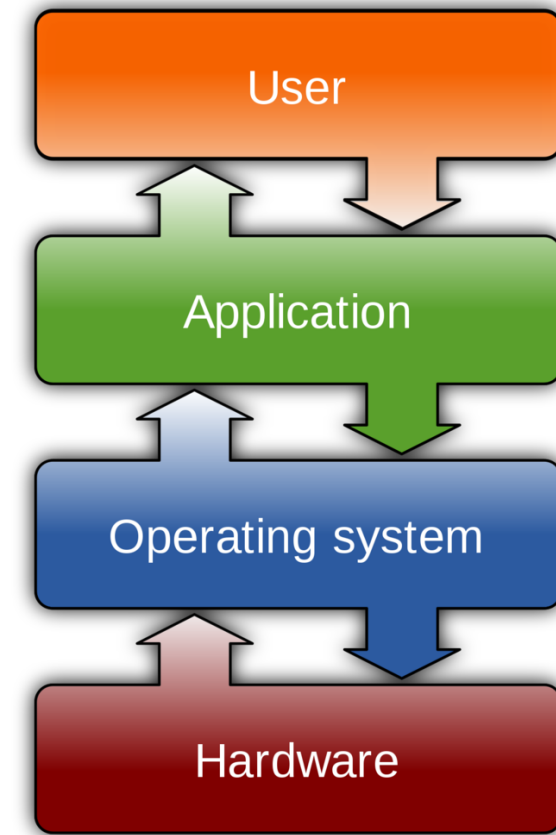
- **Booting** is the process of **loading the operating system** into main memory (RAM) so that it can begin functioning.

Bootstrap Loader

- The **bootstrap loader** is a **small program** stored in **ROM (Read-Only Memory)** that loads the **OS** into RAM.

Introduction to Operating Systems

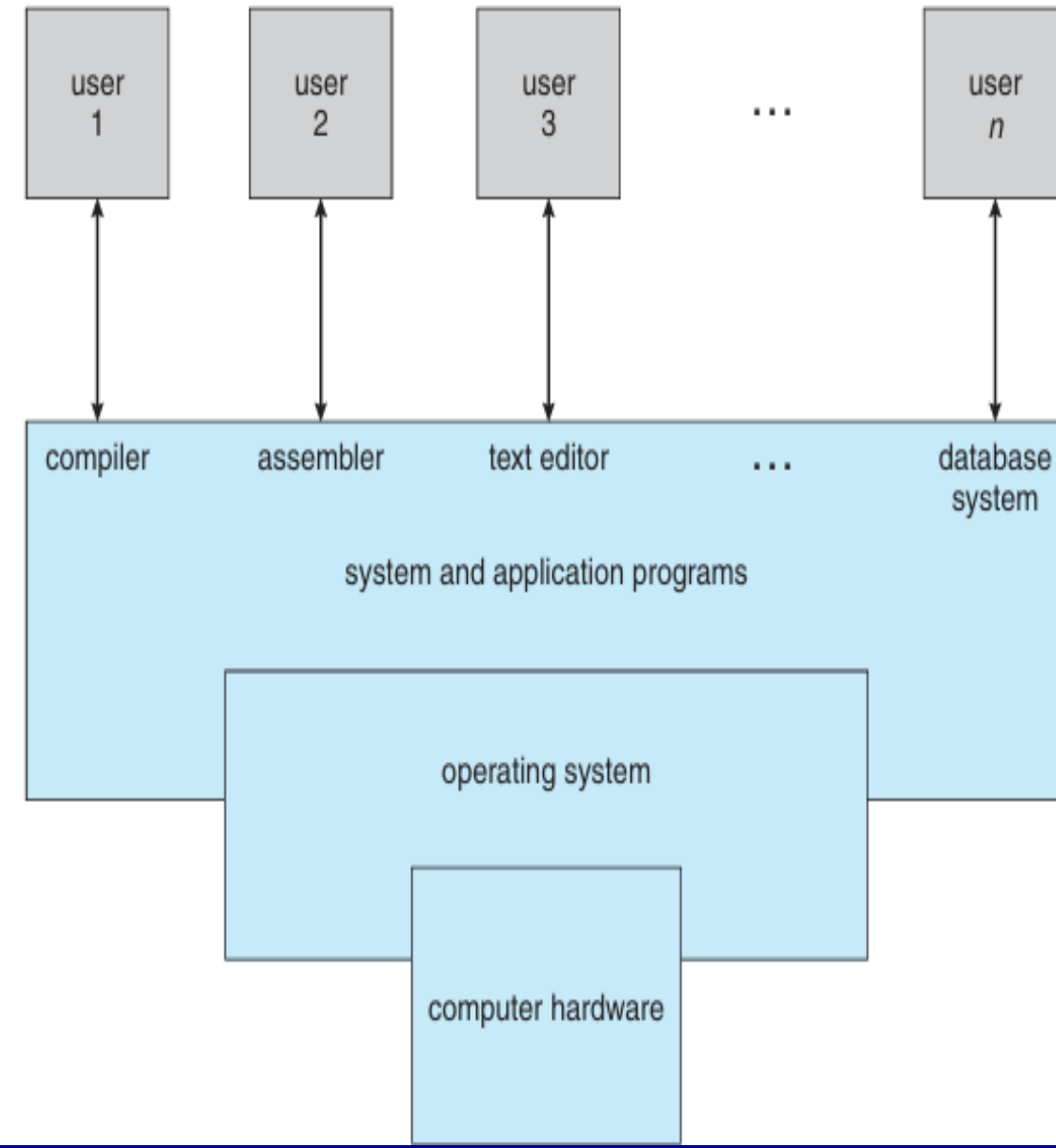
- An Operating System(OS) is a **software** that manages and handles **hardware** and **software resources** of a computing device.
- An **Operating System (OS)** is **system software** that acts as an **intermediary** between computer hardware and the user.
- Responsible for **managing and controlling** all the activities and **sharing of computer resources** among different running applications.
- Examples are Linux, Unix, Windows 11, MS DOS, Android, macOS and iOS.



What Operating Systems Do/ Operating system's role



- A computer system can be divided roughly into four components – **the hardware, the operating system, the application programs and the users.**
- The **hardware** consisting of **CPU, memory and I/O devices** provides the basic **computing resources** for the system.
- The **application programs**—such as word processors, spreadsheets, compilers, and Web browsers are developed to solve users' computing problems



What Operating Systems Do/ Operating system's role

- To fully understand what an operating system (OS) does, we can look at it from **two different perspectives**:
 1. User's View
 2. System's View

User's View:

- The user's view of the computer varies according to the interface being used.
- Based on the interface used by the user, OS has to provide either **performance**, **resource utilization** or both.

What Operating Systems Do/ Operating system's role



Case -I:

- Users sit in front of a PC, consisting of a monitor, keyboard, mouse, and system unit.
- The goal is to **maximize the work** that the user is performing.
- In this case, the operating system is designed mostly for **ease of use**.

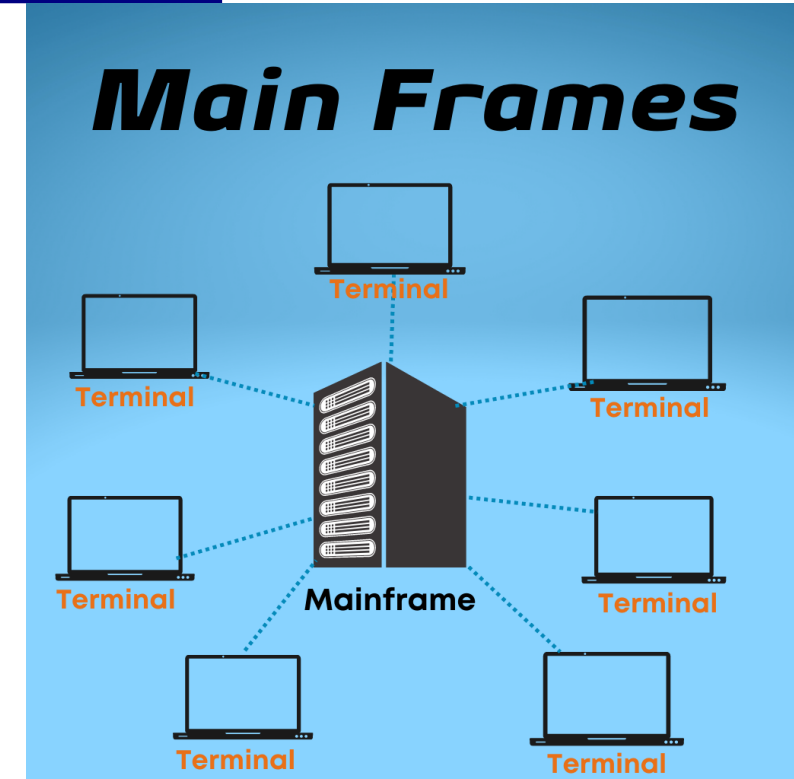


What Operating Systems Do/ Operating system's role



Case –II:

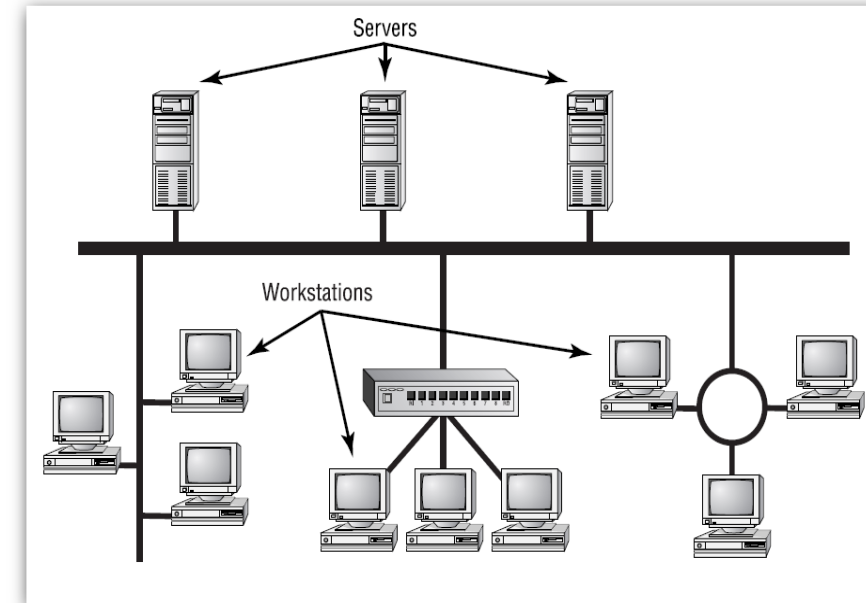
- a user sits at a **terminal** connected to a **mainframe** or a **minicomputer**.
- Other users are accessing the same computer through other terminals.
- The operating system in such cases is designed to **maximize resource utilization**— to assure that all available **CPU time, memory, and I/O** are used efficiently.



What Operating Systems Do/ Operating system's role

Case –III:

- users sit at **workstations** connected to networks of other workstations and **servers**.
- These users have dedicated resources at their disposal, but they also share resources such as networking and servers, including file, compute, and print servers.
- Therefore, their operating system is designed to provide both **performance** and **resource utilization**



What Operating Systems Do/ Operating system's role

System View:

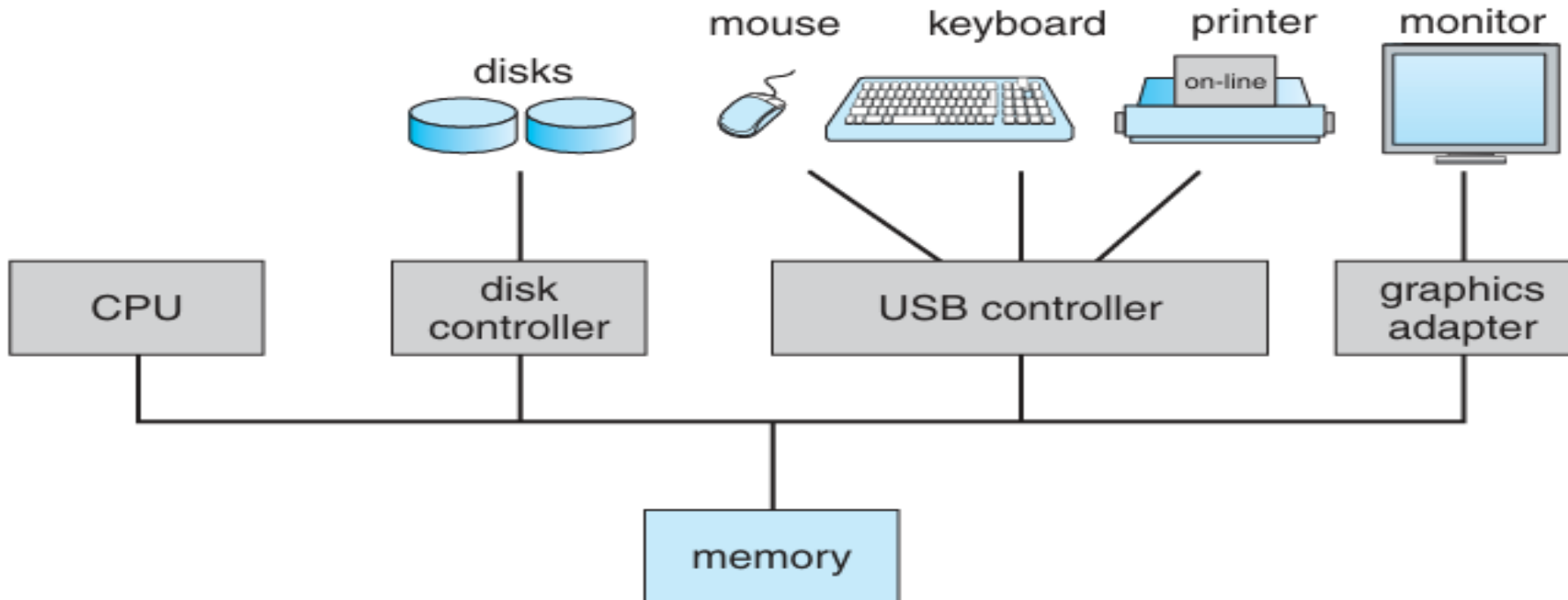
Resource Allocator/Resource Manager

- The OS manages and allocates **hardware and software resources** to various applications and users **efficiently and fairly**.
- These resources include: CPU (Processor time), Main Memory (RAM), Storage Devices (Hard drive, SSD), Input/Output Devices (Printers, USBs, etc.)

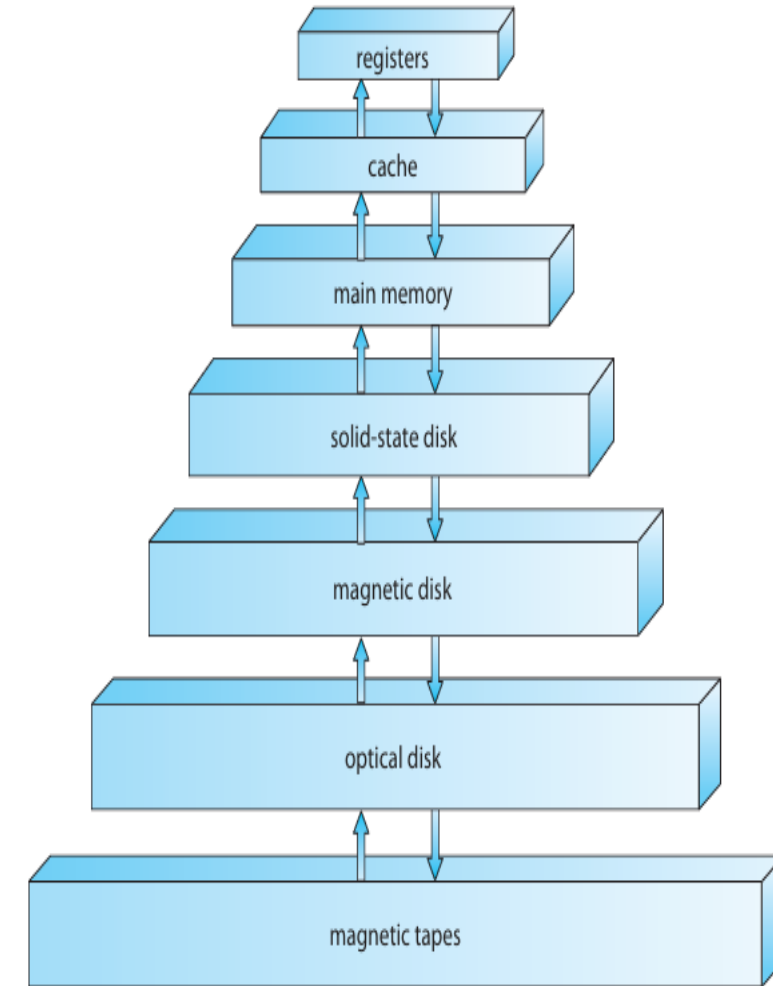
Control program.

- An OS is a **control program** that controls the execution of user programs to prevent errors and improper use of the computer.
- It is especially concerned with the operation and control of I/O devices.

- A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common bus that provides access to shared memory.
- Each device controller is in charge of a specific type of device.



- **ROM (Read-Only Memory)** is used to store **static programs** that don't change, such as the **bootstrap program**.
- The **CPU can load instructions only from memory**, so any program that needs to run must be stored in memory.
- Most programs on general-purpose computers run from **main memory**, also called **random-access memory (RAM)**.
- Main memory is usually too **small** to store all needed programs and data permanently.
- Main memory is a **volatile** storage device that loses its contents when power is turned off.
- Thus, most computer systems provide **secondary storage** as an extension of main memory and able to hold large quantities of data permanently.



Registers

- Located inside the **CPU**.
- Very small in size , Fastest and **Volatile**
- Stores temporary data, such as operands and results of operations.

Cache Memory

- Located between CPU and main memory.
- Faster than RAM but smaller in size.
- Stores frequently accessed data and instructions.

Magnetic Disk (Hard Disk Drive - HDD)

- Slower than SSDs.
- Stores large amounts of data at low cost.
- **Non-volatile** – retains data after shutdown

Solid-State Disk (SSD)

- A type of **secondary storage**.
- Faster than traditional hard drives.
- Uses flash memory (non-volatile).

I/O Devices: Keyboard, Mouse, Monitor, Disk, Printer, Network card.

- A device controller maintains some local buffer storage and a set of special-purpose registers.
- The device controller is responsible for moving the data between the I/O devices that it controls and its local buffer storage.
- Typically, operating systems have a device driver(software) for each device controller.

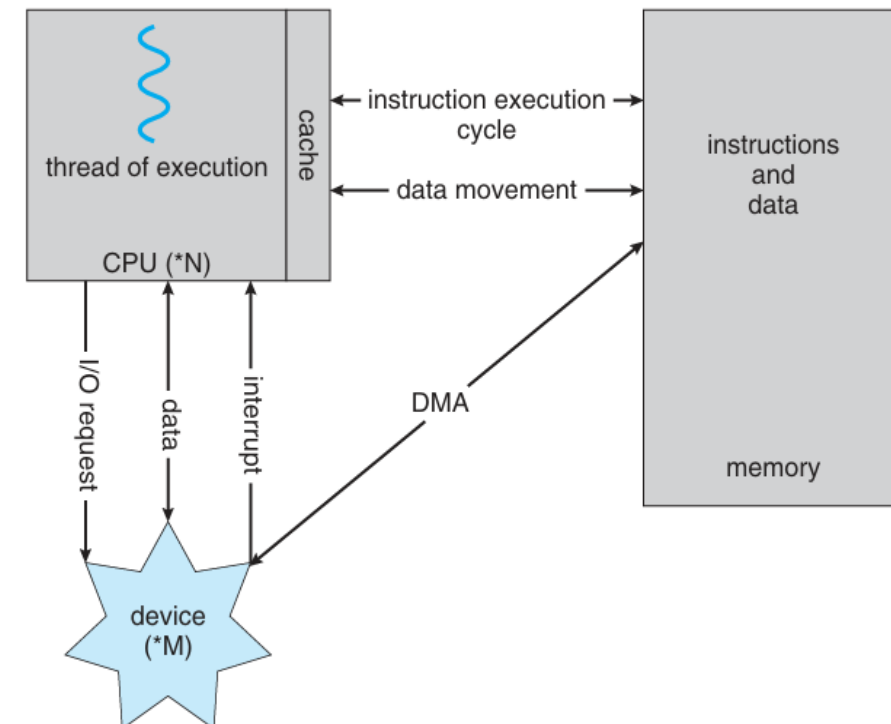
Step-by-Step I/O Operation Flow

1. To initiate an I/O operation, the device driver writes instructions (such as read or write) into the registers of the device controller
2. The device controller, in turn, examines the contents of these registers to determine what action to take (such as “read a character from the keyboard”).
3. The controller starts the transfer of data from the device(KB) to its local buffer.
4. Once the transfer of data is complete, the device controller informs the device driver via an interrupt that it has finished its operation.
5. The device driver then returns control to the operating system, possibly returning the data/status.

- This form of interrupt-driven I/O is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as disk I/O.
- To solve this problem, direct memory access (DMA) is used.
- DMA controller transfers data **directly between memory and device** without involving the CPU.

I/O Operation Flow Using DMA

- **CPU sends request to DMA controller**
→ Includes what data to transfer.
- **DMA controller sends command to I/O device**
→ Starts the data transfer.
- **Data is transferred directly between I/O device and main memory**
→ **CPU is not involved** during the transfer.
- **DMA controller sends an interrupt to CPU**
→ Informs that the transfer is complete.



1. Process Management
1. Memory Management
2. File-System Management
3. Disk management
4. Device management
5. Protection and Security
6. Networking

Process Management

- A program in execution is a process.

The operating system is responsible for the following activities

- Creating and deleting both user and system processes
- Scheduling processes on the CPUs
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication

Memory Management

- Main memory is a large array of bytes. Each byte has its own address.

The operating system is responsible for the following activities

- Keeping track of which parts of memory are currently being used and who is using them
- Deciding which processes (or parts of processes) and data to move into and out of memory
- Allocating and deallocating memory space as needed

File-System Management

- A file is a collection of related information. A file represents code and data.

The operating system is responsible for the following activities

- Creating and deleting files
- Creating and deleting directories to organize files
- Supporting primitives for manipulating files and directories
- Mapping files onto secondary storage
- Backing up files on stable (non-volatile) storage media

Disk management

- Disk Management means how the **operating system (OS)** controls and uses the **hard disk to store files, read data, and keep things organized.**
- The operating system is responsible for the following activities

Free-space management

- The OS keeps track of which parts of the disk are **empty** and **can be used** to store new files.

Storage allocation

- OS decides **where and how** to keep each file on the disk.

Disk scheduling

- Choose best order to read/write data from disk.

Device management

- **Device management in an operating system** refers to the process of controlling and coordinating input/output (I/O) devices, such as printers, disk drives, keyboards, and monitors.

The operating system is responsible for the following activities

Device Allocation

- Gives the right device to the right program or user

Deallocation

- Frees the device after use, so others can use it

Device Tracking

- Keeps track of all connected devices (mouse, printer, etc.)

Device Communication

- Sends and receives signals using **device drivers**

Error Handling

- Finds and fixes errors or problems with devices

Protection and Security

- The **Operating System (OS)** is responsible for **keeping your computer and data safe** from **unauthorized access, misuse, and malicious attacks**.

User Authentication

- Checks **who is using** the system.
- Example: Username and password login.

Authorization

- Decides **what a user can do**.
- Example: You can open your file, but not system files.

Access Control

- Controls **who can access what**.
- Example: Only admin can install new software.

Networking

- Manages data transfer across networks.
- Supports network communication protocols and remote access.

Types of Operating Systems



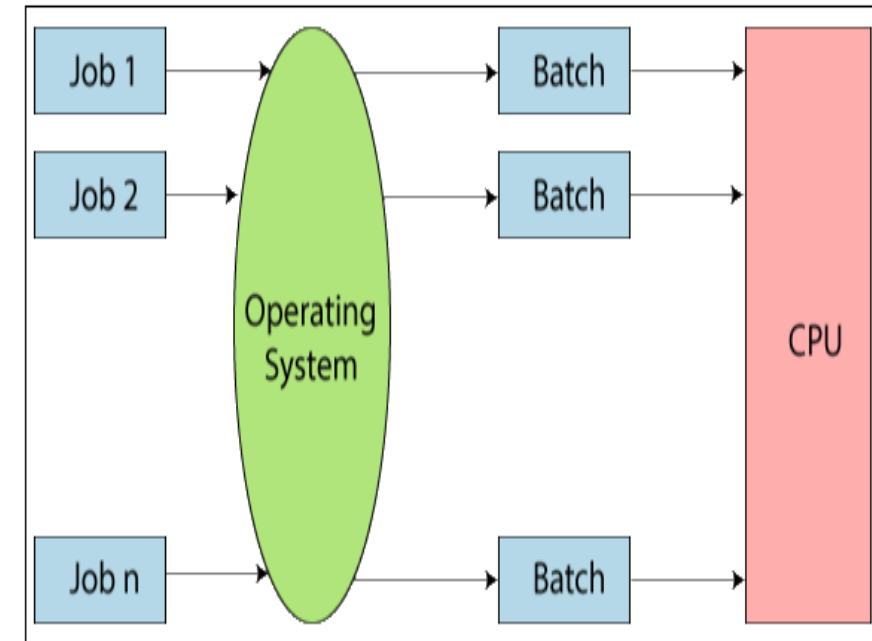
VIT-AP
UNIVERSITY

1. Batch O.S.,
2. Multi programmed O.S.,
3. Time Sharing O.S.,
4. Distributed O.S.,
5. Network OS
6. Real Time OS
7. Real Time Embedded Systems,
8. Open source O.S
9. Multiprocessor Operating System
10. Virtualization

- A **Batch Operating System** is an early type of operating system where **similar jobs are grouped together** and executed in batches **without user interaction**.
- Users prepare jobs with program code and input data using **punch cards**
- A **punch card** is a stiff paper card that encodes information using **holes punched in predefined positions**.
- Each card typically represents **one line of code or data**.

How it works:

- Jobs are **submitted to the computer operator**.
- Operator **groups similar jobs into batches**.
- The OS **executes jobs one by one** in the batch **sequentially**.
- Output is **collected and returned** to users after execution.

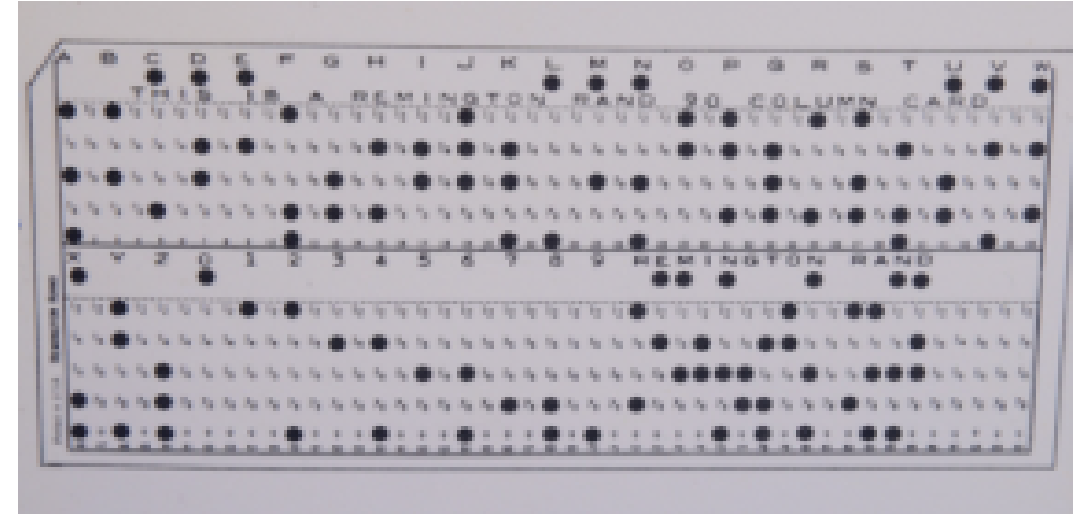


Advantages

- ✓ High CPU utilization.
- ✓ Suitable for large, repetitive jobs.

Disadvantages

- ✗ No user interaction during execution.
- ✗ Not suitable for real-time or interactive applications.



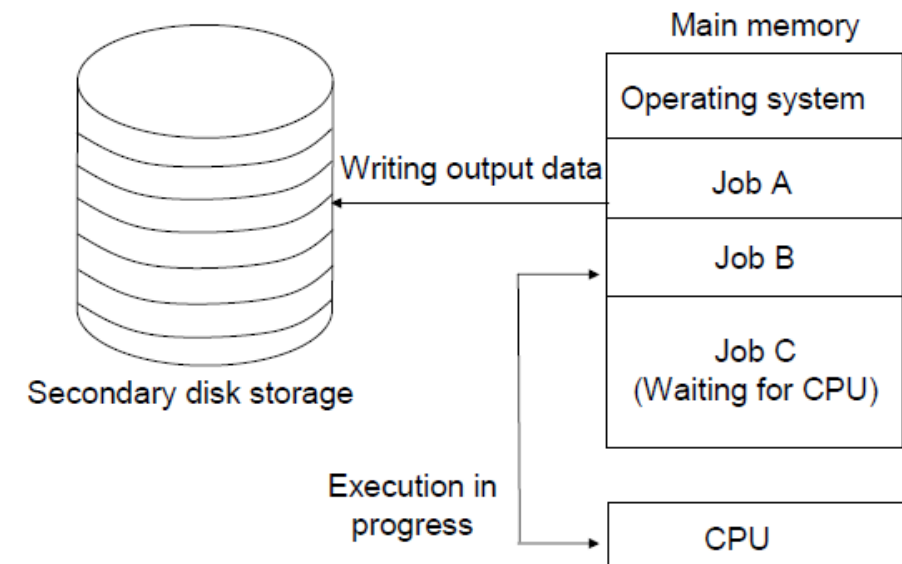
Multi programmed O.S.



- A **Multiprogrammed Operating System** is designed to run **multiple programs simultaneously**
- It keeps several programs in **main memory** at once and **switching the CPU** among them to maximize utilization.

How It Works :

1. **OS loads Multiple programs** into the **main memory**.
2. The **OS** selects one job and assign **CPU** to that job.
3. The **CPU starts running** the first program.
4. When the program **needs I/O** (like reading a file), it **stops**.
5. The OS **switches the CPU** to another program that is ready.
6. This process **keeps repeating**, so the CPU is always working.



Advantages:

- ✓ Better resource utilization (CPU, memory, I/O)
- ✓ Increases system throughput (more jobs completed per time)
- ✓ Reduces idle time of CPU

Disadvantages:

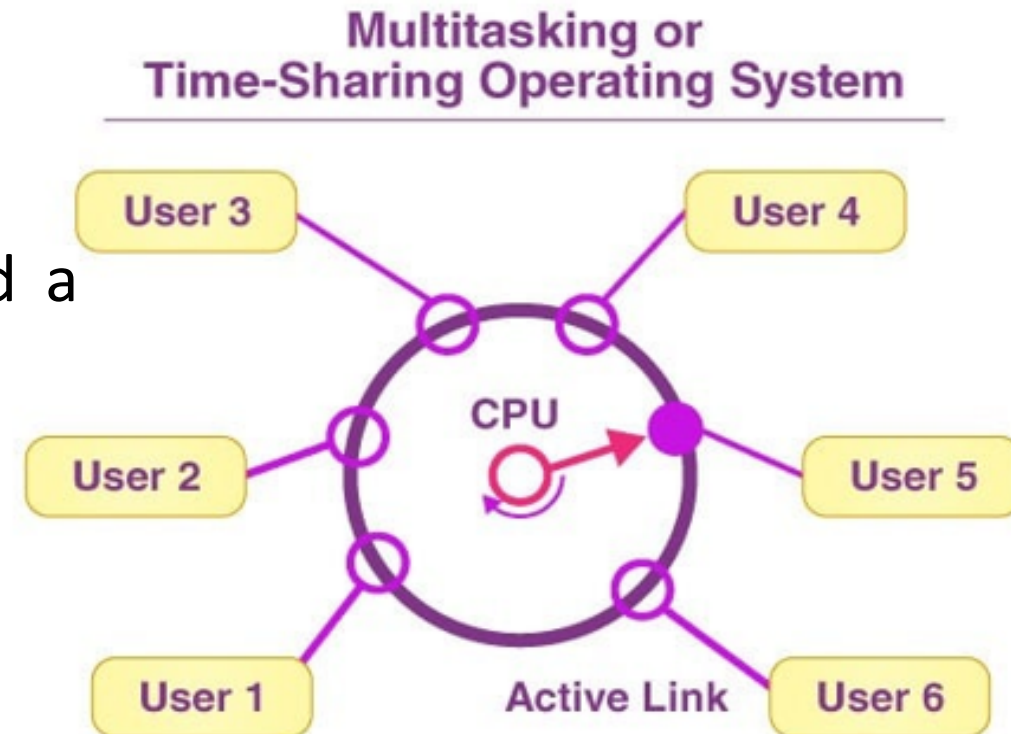
- ✗ Complex OS design (requires scheduling, memory protection)
- ✗ One job can affect another if not properly isolated

Time Sharing O.S

- It is also called a multitasking operating system.
- A **Time Sharing Operating System** allows **many users/tasks** to use a computer at the same time by **sharing CPU time**.
- Time-sharing is sharing the processor's time with multiple users to perform multiple tasks concurrently

How it works:

- Each user submits a task
- OS stores tasks in a queue
- CPU is given to one task for a short time (called a time slice).
- After the time slice, task is paused.
- Then CPU switches to another task.
- This cycle repeats quickly for all tasks



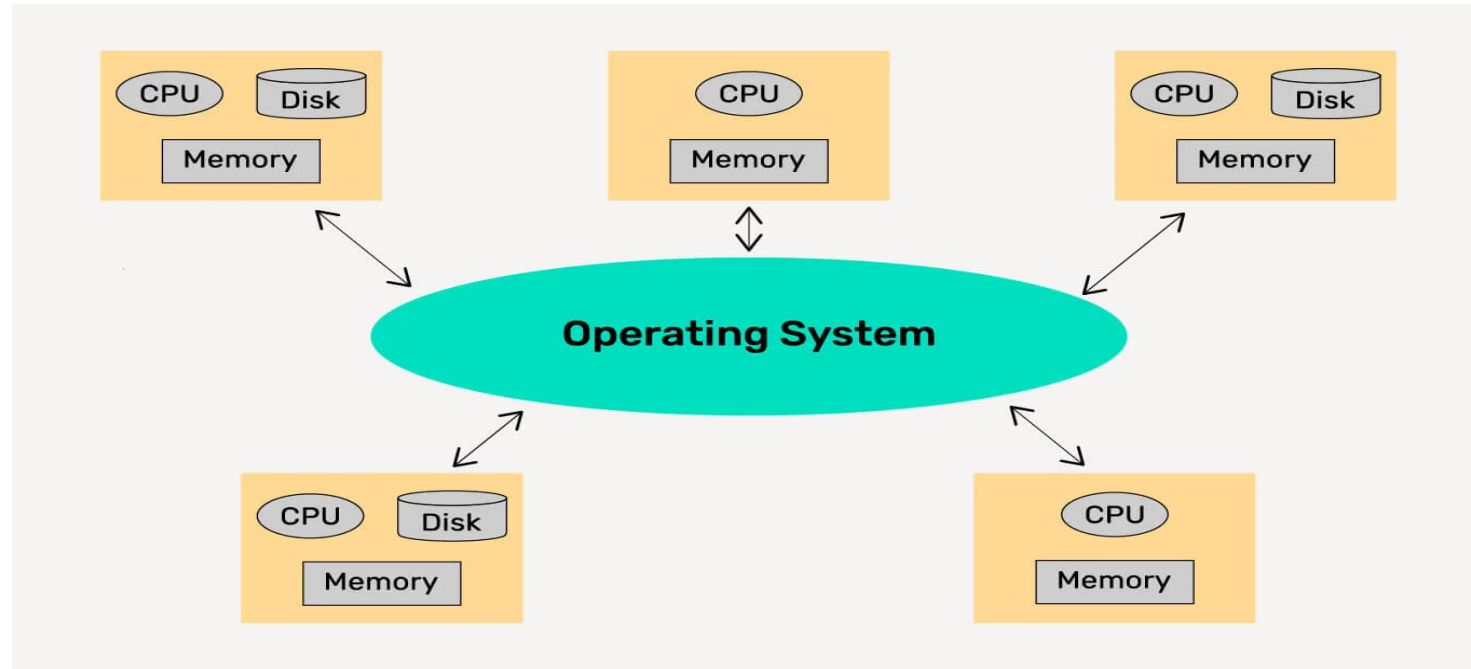
Advantages:

- Better **CPU utilization**
- **Quick response time** for users
- **Minimized idle time** for CPU

Disadvantages:

- **Complex System:** Managing many users and tasks needs a complex OS design.
- **Security Issues:** Data must be protected between users; otherwise, one user may access another's data.
- **Scheduling Overhead:** Frequent switching between tasks uses CPU time and memory.

- A **Distributed Operating System (D.O.S)** is a type of operating system that manages a group of independent computers and makes them appear to the users as a single computer.
- Multiple systems (nodes) are connected via a network.
- Each node has its own local memory and CPU.
- Distributed O.S. coordinates resources and tasks among nodes.
- It schedules jobs, manages files, and handles communication across nodes.



Advantages:

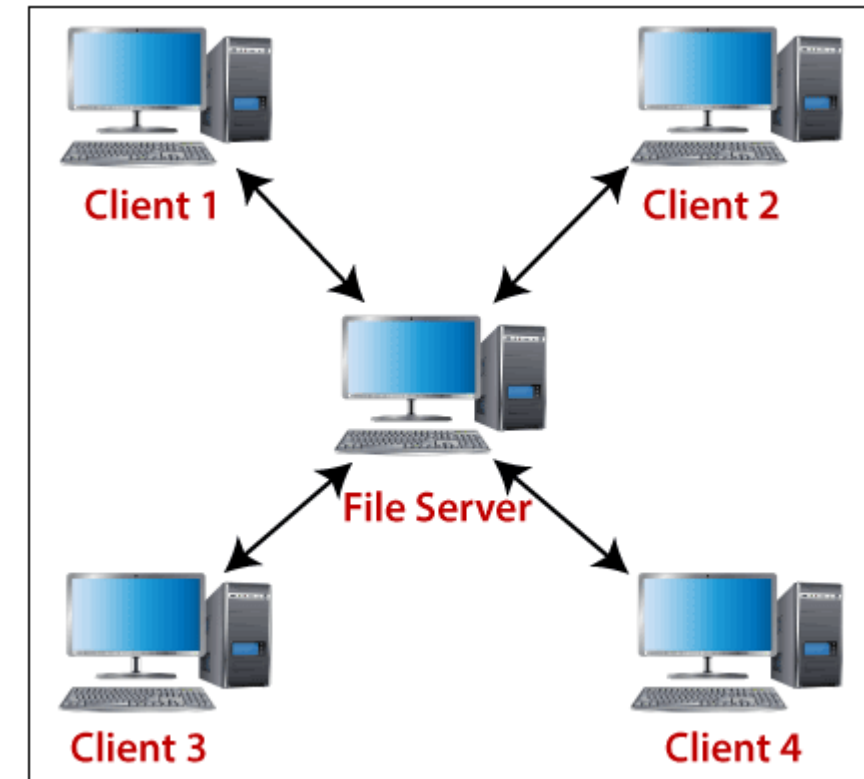
- Improved **performance** and **scalability**
- Enhanced **reliability** and **availability**
- Efficient **resource utilization**

Disadvantages:

- **Complex Design** and implementation
- **Network Dependency** – system fails if network goes down
- **Security Risks** – data travels between nodes
- **Synchronization Issues** – managing time and data consistency

Network Operating System

- Network Operating Systems **runs** on a **server**.
- A network operating system manages and coordinates network resources, allowing multiple computers to communicate and share data within a network.
- It provides essential services like file sharing, printer sharing, and network security.
- Novell NetWare and Windows Server (with Active Directory) are popular network operating systems.



- A **Real-Time Operating System** is designed to **respond to inputs or events immediately** (within a guaranteed time).
- The RTOS is suitable for domains that require precise timings and immediate responsiveness, such as industrial automation, robotics, avionics, medical devices, and telecommunications.

Examples of RTOS: VxWorks, FreeRTOS, RTLinux etc

Key Features:

Fast Response Time – completes tasks within a deadline.

Deterministic Behavior – predictable and consistent output.

Prioritized Scheduling – tasks are scheduled based on urgency.

Minimal Delays (Low Latency) – quick reaction to external inputs.

Reliability and Stability – runs continuously for long periods.

Types of Real-Time Operating System

Hard Real-Time OS: The process should be completed at the exact time.

These OS's are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable.

Example: - *Aircraft systems, Medical critical care System, etc.*

Soft Real-Time OS: The processes should be completed within the time-bound. These OS's are for applications where the time constraint is less strict.

Example: - Live stock price and Online Transaction System.

Advantages:

- Fast and predictable response
- High reliability and stability
- Efficient CPU usage
- Handles multiple tasks with priority

Disadvantages:

- Complex to design and program
- Limited multitasking compared to general-purpose OS
- Expensive for some systems
- Difficult to upgrade or modify once deployed

- A **Real-Time Embedded System** is a combination of hardware and software designed to perform **specific tasks** within **strict time constraints**.
- These systems are **embedded** into larger devices (like cars, medical machines, or washing machines) and must respond to inputs **instantly or within a fixed time**.

Working :

- System is **embedded in a larger device** (like a car or microwave).
- A **sensor or input device** sends real-world data to the system.
- The **real-time OS processes the data** quickly.
- A decision is made, and the **device responds immediately**.
- This cycle continues **in real-time** without delays.

Example Workflow (Car Airbag System):

1. Crash sensor detects a sudden impact.
2. Sends data to the embedded controller.
3. Real-time system decides to deploy the airbag.
4. Airbag inflates in milliseconds to save the passenger.

Real-Life Examples:

- **Car systems** – airbag, ABS brakes, engine control
- **Medical devices** – pacemakers, blood pressure monitors
- **Home appliances** – washing machines, microwave ovens
- **Industrial machines** – robotic arms, conveyor belts
- **Consumer electronics** – digital cameras, smart TVs

- An **Open Source Operating System** is a type of OS whose **source code is freely available** to the public.
- Anyone can **view, modify, and distribute** the code based on their needs, under an open-source license.
- If needed, users or developers can **modify the code and** can be **shared legally** (under license terms).

Popular Open Source OS Examples:

- **Linux , Android** (based on Linux kernel),**ReactOS** (Windows-like OS),**Kali Linux**

Advantages:

- Free of cost
- Faster updates and bug fixes
- Transparent and secure

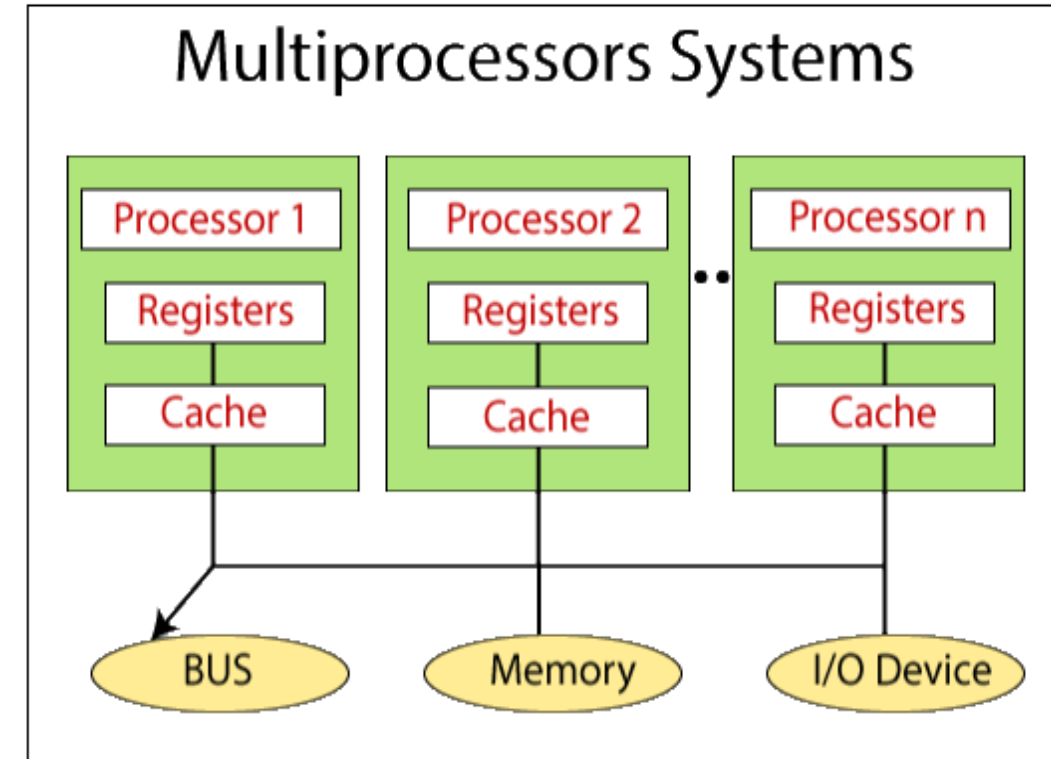
Disadvantages:

- May require technical knowledge
- Not all software is compatible
- Limited professional support (compared to paid OS)

- A **Multiprocessor Operating System** is designed to manage a **computer system with two or more processors (CPUs)** that work together to perform tasks.
- These processors share the same memory and input/output devices and communicate with each other.

How It Works (in steps):

- Computer has **2 or more CPUs**.
- The **Operating System** assigns tasks to each CPU.
- All CPUs execute their tasks **at the same time**.
- The OS **collects results** and manages them properly.



Advantages:

- Faster task execution
- Better system throughput
- More reliable – CPU failure doesn't crash the system
- Efficient use of resources

Disadvantages:

- Expensive hardware
- Complex OS design
- Requires special synchronization techniques

- **Virtualization** is a technology that allows you to create multiple Virtual Machines on a **single physical machine**, enabling better utilization of resources, flexibility, and isolation.
- **Hypervisor** refer to the **software layer** that allows multiple virtual machines (VMs) to run on a single physical machine by managing hardware resources

How does it work?

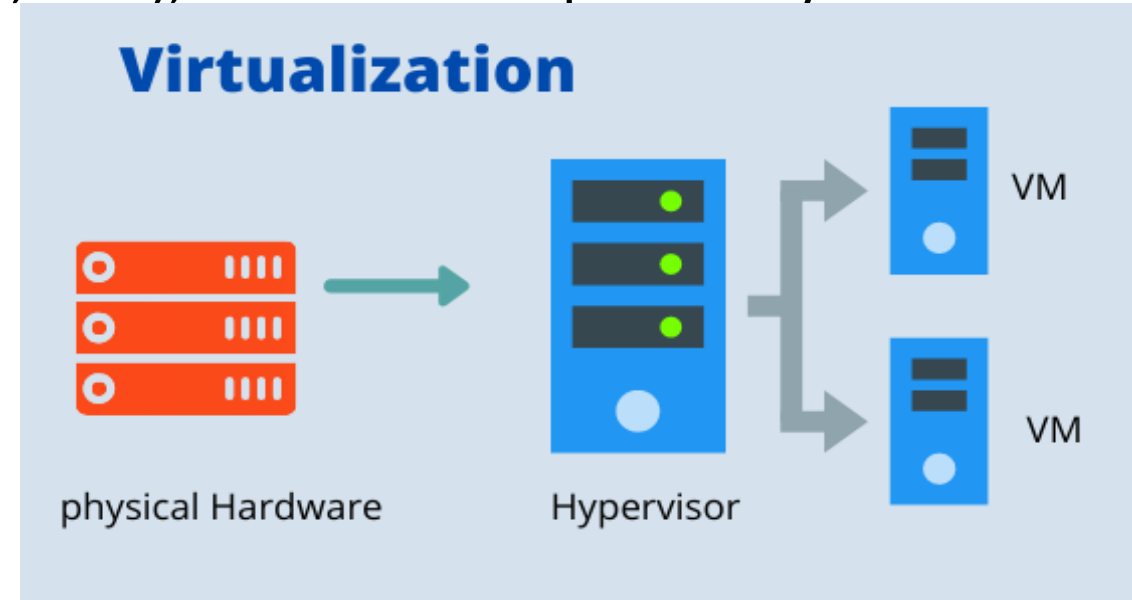
- Install a **hypervisor** on the physical machine.
- Create multiple **virtual machines**, each with its own OS.
- The hypervisor manages hardware allocation to each VM.
- All VMs **share the same physical resources** (like RAM, CPU), but work independently.

Advantages:

- Better **hardware utilization**
- **Run multiple OSes** on one system
- **Reduces hardware costs**

Disadvantages:

- Needs **powerful hardware**
- **Complex setup** for beginners
- **Security risks** if not properly managed



- Modern operating systems are **interrupt-driven** — they react to **events**.
- If there's **nothing to do** (no process, no I/O, no user input), the OS just **waits**.
- When **something happens**, it's usually signaled by:
 - An **interrupt** (from hardware), or
 - A **trap** (from software).

trap:

A **software-generated interrupt**, caused by:

Errors (e.g., divide by zero)

Requests from user programs for services (like system calls)

Dual-Mode Operation

- To ensure the proper execution of the operating system, it is essential to distinguish between operating-system code and user-defined code.
- This is achieved by using **two separate modes of operation**:

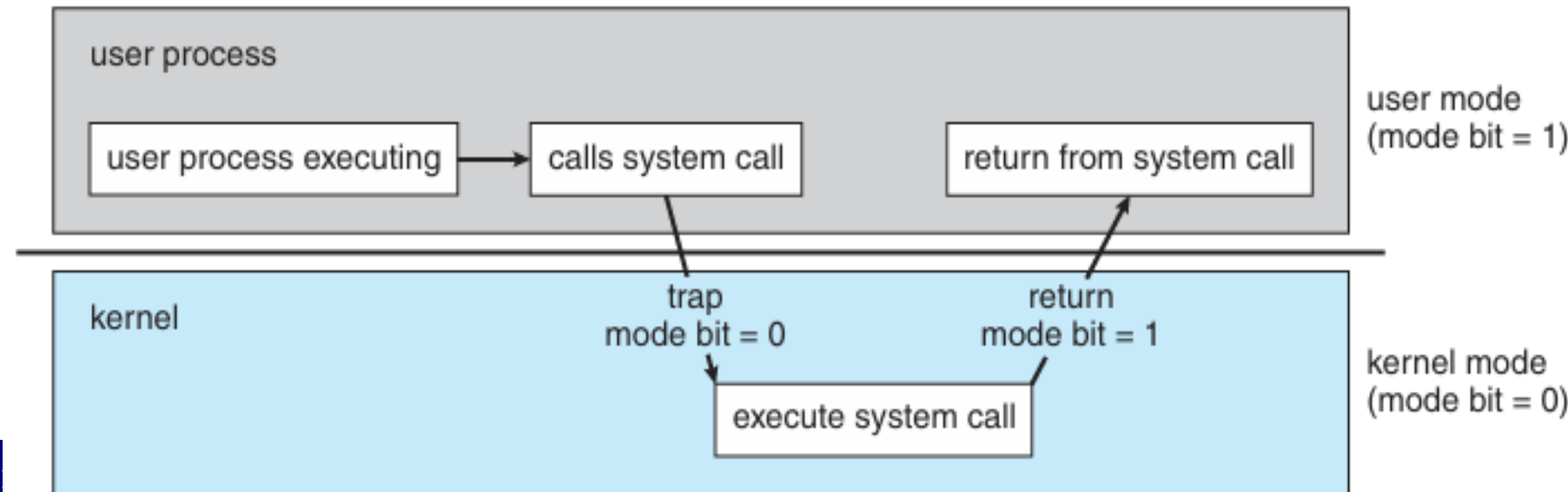
User Mode

Kernel Mode

- A special hardware-supported bit, called the **mode bit**, is used to indicate the current mode of the system:

0 for kernel mode

1 for user mode



- At **system boot time**, the hardware starts in **kernel mode**.
- During this phase, the **operating system is loaded** and then begins running **user applications in user mode**.
- Whenever a **trap** (like a system call) or an **interrupt** (from hardware) occurs, the hardware **automatically switches from user mode to kernel mode** by setting the **mode bit to 0**.
- Whenever control is returned to a **user program**, the system **switches back to user mode** by setting the **mode bit to 1**.
- This **dual-mode operation** ensures that the **operating system is protected** from **malicious actions** by user programs.
- To do this, some **privileged instructions(malicious instructions)** are only allowed in **kernel mode** because they can affect the whole system (like shutting down, changing memory).
- If a **user program** tries to run one of these privileged instructions while in **user mode**, the **hardware blocks it** and **sends a trap** to the operating system instead of letting it run.

- The idea of using **modes** can be extended to include **more than two modes**.
- In that case, the **CPU uses more than one bit** to keep track of which mode it's in.
- For example, in computers that support **virtualization**, there's often a special mode for the **Virtual Machine Manager (VMM)**.
- This is the software that controls and runs **virtual machines**.
- The **VMM mode**:
 - Has **more privileges than user programs**
 - Has **fewer privileges than the OS kernel**
- This special mode gives the **VMM just enough power to**:
 - **Create and manage virtual machines**
 - **Change the CPU's state** as needed

System Calls

- A **system call** is a way for a user-level program to request a service from the OS kernel, such as file operations, process management, or communication.

Process runs in User Mode

System Call Invocation

- The program issues a system call using a special instruction

Mode Switch: User Mode → Kernel Mode

- The CPU switches from user mode to kernel mode automatically on system call.

System Call Handler in Kernel Mode

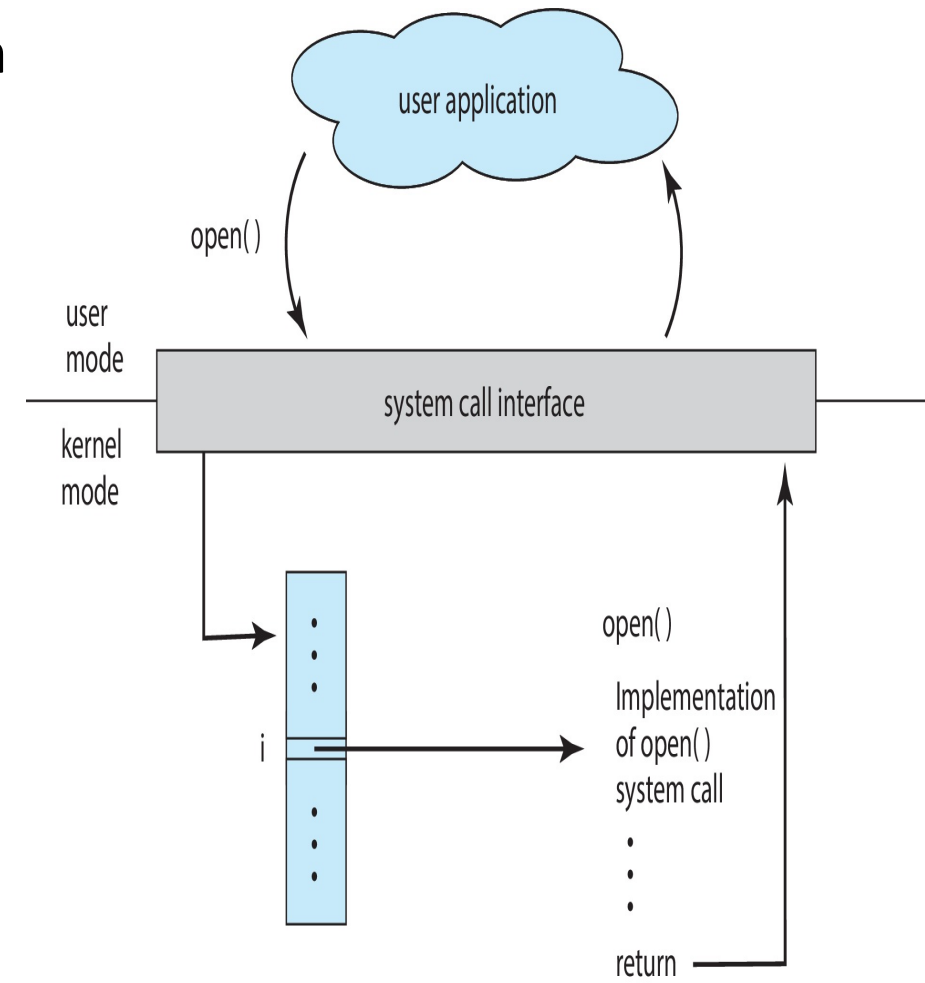
- The OS kernel executes the requested service inside the system call handler.

Return from System Call

- Once the OS finishes the requested service, it restores the saved context and switches the CPU back to user mode.

User Mode Resumes

- The application continues execution with the result of the system call



- System calls provide an interface to the services made available by an operating system.
- These calls are generally available as routines written in C and C++

Types of system calls:

Process Control

- `fork()`
- `exit()`
- `wait()`

Communication

- `pipe()`

Protection

- `chmod()`

File Manipulation

`open()`
`read()`
`write()`

Information Maintenance

- `getpid()`
- `sleep()`

- fork system call is used to create a new process.
- OS kernel creates a new process, when an existing process calls the fork () System call
- The new process is called a child process

Syntax:

```
#include<sys/types.h>
```

```
#include<unistd.h>
```

```
Pid_t fork(void);
```

Returns : '0' in child

Process ID of child to parent

'-1' on error

- fork () function is called once but returns twice and return value in the child is zero, and return value in the parent is process id of a new child.
- fork() creates a child that is duplicate of the parent process.
- The statements within a program will be executed twice once by parent and once by child.

fork()



Example-1:

```
main()
{
    printf("before fork()");
    fork();
    printf ("after fork()");
}
```

O/P :

```
before fork
After fork
before fork
After fork
```

Example-2:

```
void main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
    }
    else {
        printf("Parent process: PID = %d, Child PID = %d\n", getpid(), pid);
    }
}
```


- The exit() system call is used by a process to **terminate itself** in Unix-like operating systems.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    printf("Before Exit");
    exit(0); // Exits the process with status 0 (success)
    printf("After Exit");
}
```

O/P: Before Exit

- It is used to wait/suspend the execution of a parent process until the child process execution is completed.
- We can control the execution of the child process by calling the wait() function in the parent process.
- wait() blocks the parent until the child process terminates

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
void main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: My PID is %d\n", getpid());
    }
    else {
        // Parent process
        wait(0); // Wait for child
        printf("Parent: Parent exited after Child exited");
    }
}
```

open()



- open() is used to create a new file or open an existing file.

```
#include <fcntl.h>
```

```
int open(const char *pathname, int oflag, ... );
```

Returns: file descriptor if OK, 1 on error

- The pathname is the name of the file to open or create.
- oflag argument specifies the following

O_RDONLY	Open for reading only.
O_WRONLY	Open for writing only.
O_RDWR	Open for reading and writing.

- read() system call is used to read the data from a file.

```
#include<fcntl.h>
#include<stdio.h>
int main ( )
{
    int fd1;
    int n;
    char ch;
    fd1=open("resume",O_RDONLY);
    while((n=read(fd1,&ch,1))>0)
    {
        printf("%s",ch);
    }
}
```

- write() system call is used to write the data into a file

```
void main()
{
char buf[]="hi this is cse";
fd=open("file1",O_WRONLY);
write(fd,buf,14);
close(fd);
}
```

sleep()



- Suspends execution for a given number of seconds.

```
#include <stdio.h>
#include <unistd.h>

void main() {
    printf("Sleeping for 3 seconds ... \n");
    sleep(3);
    printf("Woke up!\n");
}
```

- The pipe() system call is used to **share data between a parent and child process** by creating a unidirectional communication channel. One process writes data to one end of the pipe, and the other reads from the other end.
- The chmod() system call in Unix is used to change the permissions (access modes) of a file or directory.